
Smart Adaptive Mode — Full Technical Architecture

Overview

Smart mode is a **multi-phase adaptive pipeline** that uses Google Cloud Vision (CV) for structured OCR, a cheap fast model (Tier-1) for text-only parsing, deterministic quality checks for routing, and an expensive powerful model (Tier-2) **only when fields actually need it**. On clean images most fields go through Tier-1 only and return in ~6-8 seconds with 1 Gemini call. Only problematic fields escalate to Tier-2.

Models and Configuration

```
Tier-1: gemini-3.1-flash-lite      (SMART_TIER1_MODEL, text only, thinking_level: mi
Tier-2: gemini-3.1-pro-preview    (SMART_TIER2_MODEL, image + crops, thinking_level
CV threshold: 0.85                (SMART_CV_CONF_THRESHOLD)
Max Tier-2 fields: 8              (SMART_MAX_TIER2_FIELDS)
```

Phase-by-Phase Pipeline

Phase 0 — Cloud Vision Rich OCR (parallel across all sides)

```
extractRichVisionLines(image) → { rawText, lines: LineRecord[] }
```

Calls `documentTextDetection` with:

- `languageHints: ['bn', 'en']` — excludes similar scripts (Arabic, Devanagari)
- `enableTextDetectionConfidenceScore: true` — per-word confidence scores
- `advancedOcrOptions: ['legacyLayout']` — heuristic layout for form-structured cards

What comes back: The CV response contains

`fullTextAnnotation.pages[].blocks[].paragraphs[].words[]`. Each paragraph becomes one `LineRecord` with:

- `text` — joined symbol text of all words in the paragraph
- `confidence` — average of word confidence scores (0-1)
- `boundingBox` — paragraph-level bounding box in pixel coordinates

- `id` — `front_N` or `back_N`
- `side` — which card side

Key insight: CV groups contiguous text into blocks. If a label and its value are separated by a large horizontal gap (e.g. 118 px vs normal 35–40 px), CV splits them into **separate blocks** → separate `LineRecord`s. This split is the primary signal for gap detection.

Phase 1 — Gap Detection (zero API calls, pure geometry)

```
detectLabelValueGaps(allLines) → GapReport[]
```

Scans `LineRecord[]` for inline-label fields where the label and value are in separate blocks:

Label catalog (inline layout only — label and value on the same row):

Keyword	Field
মাতা	<code>motherNameBn</code>
পিতা	<code>fatherNameBn</code>
স্বামী	<code>fatherNameBn</code>
নাম	<code>nameBn</code>
ID NO	<code>nidNumber</code>
NID No	<code>nidNumber</code>

For each label-matching line, `isLabelOnlyLine()` checks: after stripping the keyword and any trailing colon/whitespace, is there any remaining content? If not — it's a label-only block.

Gap measurement:

1. Compute the label block's right edge X and center Y
2. Find all other lines within `lineHeight × 0.7` Y-tolerance and within 500 px horizontally to the right
3. Take the nearest candidate
4. `Gap = candidate.left - label.right`
5. If `gap > 80 px` → emit `GapReport`

`GapReport` contains: `{ field, side, labelBbox, valueBbox, fullCropBbox, gapPx }`

The `fullCropBbox` is the union of label + value bounding boxes — this is the exact region sent to Tier-2.

Why 80 px? From the live image inspection:

- Normal label → value gaps: 35–40 px
- Flash-obiterated gap: 118 px
- Threshold at 80 px sits cleanly between them

Phase 2 — Glare Detection (luminance grid, ~24 ms)

```
detectGlare(buffer) → GlareReport { regions: BoundingBox[], coverage, width, height }
```

- Converts to grayscale, downsamples to a `ceil(W/32) × ceil(H/32)` cell grid
- Flags cells with mean luminance ≥ 210 (lowered from 240 to catch JPEG-softened flash spots)
- Flood-fill merges adjacent flagged cells into bounding boxes
- `coverage` = fraction of flagged cells / total cells
- Threshold for "side has glare": **0.5%** coverage

Used downstream to decide whether to apply CLAHE enhancement before cropping. Also emits `glare_<field>` tags to `qualityIssues` when a field's bbox overlaps a detected glare region.

Phase 3 — Tier-1 Text Parse (flash-lite, no images)

```
geminiClient().interactions.create({
  model: 'gemini-3.1-flash-lite',
  generation_config: { thinking_level: 'minimal', thinking_summaries: 'none', temperature: 0.2, response_format: { type: 'text', mime_type: 'application/json' } },
  input: [formatLines(allLines)]
})
```

Input format (one line per CV paragraph):

```
[front_0] side=front conf=0.981 text="গণপ্রজাতন্ত্রী বাংলাদেশ সরকার"
[front_4] side=front conf=0.983 text="নাম : ইফফাত আরা বেগম"
[front_6] side=front conf=0.942 text="মাতা :"
[front_7] side=front conf=0.982 text="আরা বেগম"
...
```

Expected output — a wrapper JSON with two keys:

```
{
  "extraction": { <standard NID result object> },
  "fieldSources": {
    "motherNameBn": {
      "side": "front",
      "lineIds": ["front_7"],
      "minConfidence": 0.982,
      "needsVision": false,
      "reason": "clear OCR"
    }
  }
}
```

`fieldSources` lets Tier-1 report which CV lines it used for each field and whether it's confident. If the model returns flat NID JSON (no wrapper), the parser auto-detects and wraps it. If parsing fails entirely, all fields fall back to Tier-2.

Why flash-lite? Text-only, no images, minimal thinking. Typically 5–8 s, cheap. Most clean images pass entirely through this tier.

Phase 4 — Deterministic Routing

```
routeSmartFields(tier1, cvConfThreshold=0.85, providedSides) → SmartRoutingDecision
```

For each of the 11 NID fields, assigns one of three actions:

Action	Condition
<code>pass</code>	<code>confidence='high'</code> , <code>needsReview=false</code> , <code>needsVision=false</code> , no validation issues, <code>source.minConfidence ≥ 0.85</code>
<code>verify</code>	Low confidence, failed validation, low CV confidence, <code>needsVision=true</code> , or expected-but-missing
<code>absent</code>	Field not expected on provided sides for this card variant

Validators that can trigger `verify` :

- `validateNidNumber` — must be 10/13/17 digits (smart=10 only)
- `validateDateField` — must match `DD MMM YYYY`
- `validateBloodGroup` — must be in {A+, A-, B+, B-, O+, O-, AB+, AB-}
- `validateBanglaFields` — Bengali fields must contain Unicode range U+0980–U+09FF
- `validateExpectedFields` — expected-but-missing fields on provided sides
- `validateCardType` — card type must match

- `isFieldExpectedOnSides` — expected-but-null tiers on provided sides → `verify` not absent

Gap-detection upgrade (post-routing): After base routing, any field that was `pass` but has a gap report is **promoted to `verify`**. This is the key override — the gap is structural evidence of missing text that Tier-1 could not have seen.

Cap: `verify.slice(0, maxTier2Fields)` — max 8 fields to Tier-2 per run.

Fast path: If `verify.length === 0` and Tier-1 succeeded, return immediately with Tier-1's extraction. No Tier-2 call, no image upload, 1 Gemini call total.

Phase 5 — Tier-2 Input Preparation

For each field in `verify`, one of three crop paths:

A. Gap-detected path (field has a `GapReport`): Generates **3 crop variants in parallel** from `fullCropBbox` (label + gap + value) with 15% padding:

Variant	Processing	Purpose
RAW	None	Original pixels baseline
AGGRESSIVE	<code>clahe({width:20,height:20,maxSlope:5}) + gamma(2.4) + normalise</code>	Amplifies partial strokes inside narrow glare strip
NEGATED	<code>negate() + normalise</code>	Inverts image — glare becomes dark, surviving strokes become bright

All 3 are uploaded to Files API in parallel and sent to Tier-2 with labels like `[GAP DETECTED] field=motherNameBn, gap=118px — NEGATED crop`.

B. Standard path with known bbox (field has `source.lineIds`): Crops the CV-reported bounding box. If bbox overlaps a glare region → apply `enhanceForGlareRecovery` (CLAHE 50px + normalise) before cropping. Emits `glare_<field>` to `qualityIssues`.

C. Expected-but-missing path (field expected on side but CV found nothing): If side has glare coverage > 0.5% → uploads full CLAHE-enhanced side image. Emits `glare_<field>`.

Original full-side images are always uploaded to Files API for context. All uploads go to

Gemini Files API (48 h auto-expiry, referenced by URI), and are deleted immediately after the Tier-2 response.

Phase 6 — Tier-2 Visual Verification

```
geminiClient().interactions.create({
  model: 'gemini-3.1-pro-preview',
  generation_config: { thinking_level: 'high', thinking_summaries: 'none', temperature: 0.2,
    response_format: { type: 'text', mime_type: 'application/json', schema: NID_JSON_SCHEMA },
  },
  input: [
    { text: TIER_1_EXTRACTION + FIELDS_TO_VERIFY + DETECTED_QUALITY_ISSUES,
      ...originalUris (full side images),
      ...cropParts (labeled crops + variants)
    }
  ]
})
```

`NID_JSON_SCHEMA` is enforced via `response_format.schema` — ensures the model always returns parseable JSON without markdown fences.

Prompt sections:

1. NID card format reference (all 3 variants: laminated/smart/temporary, field-label mappings, সাক্ষী fatherNameBn rule)
2. Bengali OCR reconstruction rules (12 categories: conjuncts, reph, Chandrabindu, matras, character confusions, NID vocabulary)
3. Confidence rules
4. Glare/capture recovery instructions
5. Gap-detected fields instructions — explicitly guides the model to:
 - Examine all 3 variants
 - Look for `৳` at the right edge of the dark zone in the NEGATED variant
 - Note that "ম মমম ৳" ending in `৳` is the most common Bangladeshi mother-name prefix

Phase 7 — Result Merge

```
// verifyFieldSet = Set of fields that went to Tier-2
// PASS fields: restore Tier-1 values (prevent regression)
// VERIFY fields: use Tier-2 values (it had image context)

for (const field of FIELD_KEYS) {
  if (!verifyFieldSet.has(field)) {
    mergedExtraction[field] = tier1.extraction[field]; // restore Tier-1
  }
}
```

```
}
```

This merge prevents Tier-2 from overwriting correctly-extracted fields it was never shown. Example: `fatherNameBn` was `pass` → Tier-2 never saw it → its Tier-1 value is preserved regardless of what Tier-2 returned.

Quality issues merge:

```
qualityIssues = dedupe([...strategyDetected, ...modelReported])
```

Strategy detects `gap_X` / `glare_X` from geometry. Model confirms or adds additional tags.

`withReviewList()` — post-processing:

- Runs `crossFieldCheck` (`cardType` ↔ `nidNumber` digit count, `placeOfBirth` only on smart, `validUntil` only on temporary, Bengali fields contain Bengali script)
- Adds any failing fields to `fieldsNeedingReview`
- Downgrades `overallConfidence` from `high` → `medium` if any review fields exist

Output

```
{
  "extraction": {
    "cardType": "laminated",
    "nidNumber": { "value": "...", "confidence": "high", "needsReview": false },
    ...
    "motherNameBn": { "value": "রিফাত আরা বেগম", "confidence": "low", "needsReview": true },
    "overallConfidence": "low",
    "fieldsNeedingReview": ["motherNameBn"],
    "qualityIssues": ["gap_motherNameBn", "glare_motherNameBn"]
  },
  "visionOutputs": [{ "side": "front", "rawText": "...", "timingMs": 941 }],
  "timing": {
    "vision_front": "941ms",
    "glare_detection": "32ms",
    "gemini_tier1_text_parse": "5.86s",
    "files_upload_originals": "3.68s",
    "enhance_image": "1.69s",
    "crop_field": "21ms",
    "files_upload_crops": "4.03s",
    "gemini_tier2_visual_verify": "35.60s"
  },
  "geminiCallCount": 2,
  "tokenUsage": { "inputTokens": 11272, "outputTokens": 1301, "thoughtTokens": 341 }
}
```

Decision Tree Summary

CV rich OCR

- gap detection (geometry, free)
- glare detection (luminance grid, ~30 ms)
- Tier-1 parse (flash-lite, text only, ~6 s)
- routing per field:
 - pass? → return immediately (1 call total, ~7 s)
 - verify? → prepare crops:
 - gap field? → 3 variants (raw + aggressive + negated)
 - glare bbox? → CLAHE-50 enhanced crop
 - glare side? → CLAHE-50 full-side image
 - other? → raw CV bbox crop
- Tier-2 pro (images + crops, ~35 s, 1 more call)
- merge (Tier-1 for PASS, Tier-2 for VERIFY)
- cross-field check → finalize